

# W2 教程：C 内存与调试

---

sys-netsec-lab · 阶段一 · 系统基础与 TCP 网关 日期：2026-08-03 ~ 08-10 前置：W1 编译链与构建系统

---

## 目录

---

1. 第一阶段：为什么内存错误是 C 程序员的必经之路
  2. 第二阶段：内存布局——你的变量住在哪里
  3. 第三阶段：五类内存缺陷（手写 Bug，亲眼看到后果）
  4. 第四阶段：三件武器——ASan / UBSan / Valgrind
  5. 第五阶段：实战任务
  6. 参考速查表
  7. 常见面试追问
- 

## 第一阶段：为什么内存错误是 C 程序员的基本功

---

## 1.1 先看一个真相

大部分安全漏洞的根源是内存错误。Google 统计过 Chrome 的严重漏洞，**70% 以上是内存安全问题**（use-after-free、buffer overflow 等）。

对于网络方向的工程师，这个问题更致命：你的代码跑在数据面上，处理的是不受信任的网络输入。一个缓冲区溢出不只是 crash——它可能是远程代码执行的入口。

**面试官怎么考你：**

- 不是问"什么是内存泄漏"——太简单
- 而是给你一段有 Bug 的代码，问："这段代码在什么情况下会出问题？用 Valgrind 会报什么？"
- 追问："如果你的程序已经部署在生产环境，没有 Valgrind，你怎么定位这个问题？"

## 1.2 学习目标

这周结束时，你应该能：

1. 看到一段 C 代码，**在运行之前**就指出可能的内存问题
2. 写出五类内存缺陷的**最小复现代码**
3. 用 ASan / Valgrind 精准定位问题行号
4. 解释：为什么某些内存错误"偶尔 crash、偶尔不 crash"
5. 设计测试用例来**稳定触发**间歇性内存 Bug

## 第二阶段：内存布局——你的变量住在哪里

### 2.1 进程虚拟地址空间





**面试常考：**说出下面每个变量在哪：

```
#include

int global_init = 42;      // → .data 段

int global_zero;          // → .bss 段 (自动初始化为 0)

const char* msg = "hello"; // msg 在 .data, "hello" 在 .rodata

void func(int param) {    // param → 栈

    int local = 7;        // local → 栈

    static int counter = 0; // counter → .data 段 (static 改变生命周期, 但不改变存储位置)

    char buf = malloc(64); // buf → 栈, buf (64 字节) → 堆

}
```

**pmap 命令**可以查看一个运行中进程的内存映射：

```
pmap -x
```

你会看到每段内存的起始地址、大小、权限（rwx）

|  |
|--|
|  |
|  |
|  |

## 2.2 栈 vs 堆的核心区别

|      | 栈 Stack                            | 堆 Heap                             |
|------|------------------------------------|------------------------------------|
| 分配方式 | 编译器自动分配/释放                         | 程序员手动 malloc/free                  |
| 大小限制 | 通常 8MB（ <code>ulimit -s</code> 查看） | 受系统内存和 swap 限制                     |
| 速度   | 极快（移动栈指针即可）                        | 较慢（需要查找空闲块）                        |
| 碎片   | 无碎片                                | 可能产生碎片                             |
| 生命周期 | 函数返回时自动释放                          | 直到 free                            |
| 典型错误 | 返回局部变量地址                           | 忘记 free、double free、use-after-free |

## 第三阶段：五类内存缺陷

**学习原则：**你需要亲手写出每类 Bug，让它 crash，然后修复。只看不写是学不会的。

### 3.1 缓冲区溢出 (Buffer Overflow)

**定义：**向分配的内存边界之外写入数据。

```
// BUG #1: 栈溢出

void stack_overflow() {

    char buf[8];

    strcpy(buf, "this string is way too long for 8 bytes");

    // buf 只能装 8 字节 (包括 '\0' 只能装 "1234567") ,

    // 但 strcpy 不会检查, 直接写穿 buf, 覆盖栈上相邻数据

}

// BUG #2: 堆溢出

void heap_overflow() {

    char* buf = malloc(8);

    strcpy(buf, "too long");

    // 覆盖了堆上的元数据 (chunk header) ,

    // free 时可能 crash ("corrupted double-linked list")

}

// BUG #3: off-by-one (最难发现的溢出)

void off_by_one() {

    char buf[8];

    for (int i = 0; i <= 8; i++) { // <= ! 应该是 <

        buf[i] = 'A';

    }

}
```

```
// buf[8] 越界，但 buf[0-7] 都正常

// 如果 buf[8] 恰好落在对齐填充上，可能不 crash.....

}
```

**为什么有时不 crash：**写穿的区域可能是对齐填充、未被使用的栈空间、或者碰巧没有关键数据。这反而是危险的——Bug 存在但不崩溃，直到生产环境某个特定输入触发。

### 3.2 使用已释放的内存（Use-After-Free）

**定义：**free 之后继续使用指针。

```
// BUG #4: Use-After-Free

void use_after_free() {

    char* p = malloc(32);

    strcpy(p, "important data");

    free(p);

    // p 现在是悬空指针 (dangling pointer)

    printf("%s\n", p); // 未定义行为!

    // 可能还打印出数据 (free 后内存不一定立刻被清零/回收)

    // 可能打印乱码 (内存已被重新分配给别人)

    // 可能 crash (内存页已归还给 OS)

    // 更危险的场景:

    char* q = malloc(32); // q 可能拿到刚 free 的同一块内存
```

```
strcpy(q, "malicious");

printf("%s\n", p); // p 现在打印的是 "malicious"! 逻辑完全混乱

}
```

### 为什么 UAF 是最危险的内存错误之一：

- 它不一定会 crash（区别于 NULL 解引用）
- 被释放的内存可能很快被 malloc 重新分配出去
- 攻击者可以通过"堆喷"（heap spraying）精准控制这段内存的内容

## 3.3 内存泄漏（Memory Leak）

**定义：** malloc 之后从不 free，导致内存持续增长。

```
// BUG #5: 内存泄漏

void memory_leak() {

    while (1) {

        char* buf = malloc(1024);

        // 处理 buf ...

        // 忘记 free(buf);

        // 每次循环泄漏 1KB，无限循环 = 无限泄漏

    }

}

// 更隐蔽的泄漏：函数提前返回
```



```
char read_config(const char path) {

    char* file_buf = malloc(4096); // 读文件用的缓冲

    FILE* f = fopen(path, "r");

    if (!f) {

        return NULL; // ← 泄漏! file_buf 没 free

    }

    // ... 正常处理 ...

    free(file_buf);

    fclose(f);

    return result;

}

// 正确做法: 要么统一在出口 free, 要么用 goto cleanup 模式
```

#### 检测方法:

- Valgrind 的 memcheck 会明确告诉你 "N bytes in M blocks are definitely lost"
- 观察进程 RSS (Resident Set Size) : `ps aux | grep prog`
- 长期运行测试: `while true; do ./prog; done` 观察内存趋势

### 3.4 未初始化的内存 (Uninitialized Memory)

**定义:** 读取从未被赋值的变量或 malloc 出来的内存。

```
// BUG #6: 栈上的未初始化变量
```

```
void uninit_stack() {  
  
    int important_flag;  
  
    if (important_flag == 42) { // 未初始化就读取!  
  
        printf("this should never print\n"); // 但你无法保证它不 print  
  
    }  
}
```

```
// BUG #7: malloc 不初始化
```

```
void uninit_heap() {  
  
    int arr = malloc(100 * sizeof(int));  
  
    // malloc 不保证清零! 内容是什么?  
  
    // - 可能全是 0 (刚开机时)  
  
    // - 可能是上次 free 的数据 (信息泄漏风险!)  
  
    // - 可能是随机垃圾  
  
    int sum = 0;  
  
    for (int i = 0; i < 100; i++) {  
  
        sum += arr[i]; // 对未初始化值求和, 结果不可预测  
  
    }  
}
```

```
// 正确做法:
```

```
// 栈上: 声明时初始化 int flag = 0;
```

```
// 堆上: calloc 代替 malloc (自动清零)

//          or malloc + memset(buf, 0, size)
```

**Valgrind 的警告:** Conditional jump or move depends on uninitialised value(s)

**为什么危险:**

- 信息泄漏: free 后的数据可能包含密钥、密码等敏感信息
- 不可复现的 Bug: 有时能跑, 有时不能, 取决于当时那块内存的内容

### 3.5 双重释放 (Double Free)

**定义:** 对同一块内存调用两次 free。

```
// BUG #8: Double-Free

void double_free() {

    char* p = malloc(32);

    free(p);

    free(p); // ← 第二次 free! 未定义行为

    // glibc 通常会报:

    // free(): double free detected in tcache 2

    // 或者

    // corrupted double-linked list

    // 或者

    // 直接 abort()
```

```
}

// BUG #9: 更隐蔽的 double free — 别名指针

void double_free_alias() {

    char* a = malloc(64);

    char* b = a;    // b 和 a 指向同一块内存

    free(a);

    free(b);        // 程序员忘了 b 是 a 的别名

}
```

### 3.6 五类缺陷总结

| 类型             | 一句话            | 会立刻 crash 吗 | 关键工具                |
|----------------|----------------|-------------|---------------------|
| 缓冲区溢出          | 写到分配的范围外面      | 不一定         | ASan                |
| Use-After-Free | free 后还在用      | 不一定         | ASan + Valgrind     |
| 内存泄漏           | malloc 后不 free | 不会（但慢慢死）    | Valgrind            |
| 未初始化内存         | 读了没写的值         | 不一定         | Valgrind (Memcheck) |
| 双重释放           | free 两次        | 通常会         | ASan + glibc 自身检测   |

---

## 第四阶段：三件武器

---

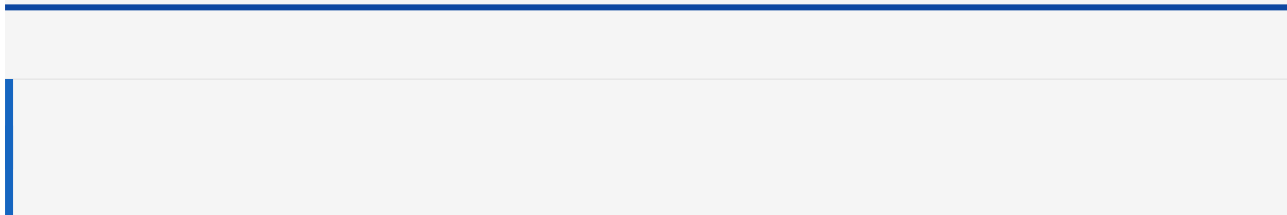
### 4.1 AddressSanitizer (ASan)

**原理：**编译时在每次内存访问前后插入检查代码。它在分配的内存周围插入"红色区域" (poisoned redzone) ，任何对红区的访问都会被立即检测到。

**启用：**

```
gcc -fsanitize=address -g -O1 program.c -o program
```

└─ 启用 ASan    └─ 调试符号    └─ 不要-O2 以上(会优化掉一些检测)



运行时可以加环境变量控制行为：

```
ASAN_OPTIONS=detect_leaks=1:abort_on_error=1 ./program
```

它检测什么：

-  Stack buffer overflow — 栈上越界
-  Heap buffer overflow — 堆上越界
-  Use-after-free — 释放后使用
-  Double-free — 双重释放
-  Memory leaks — 内存泄漏（需要 `detect_leaks=1`，Linux 默认开启）

输出示例：

```
=====
==12345==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x...
READ of size 1 at 0x... thread T0
    #0 0x... in main /tmp/test.c:8
    ...
0x... is located 0 bytes to the right of 8-byte region
allocated by thread T0 here:
    #0 0x... in malloc
    #1 0x... in main /tmp/test.c:6
```

**关键：**ASan 不仅告诉你出错了，还告诉你：

- 什么类型的错误 (heap-buffer-overflow / stack-use-after-free 等)
- 哪个文件哪一行触发的问题
- 这块内存是在哪里分配的

**性能代价：**约 2x 慢，约 2-3x 内存。所以只在测试/调试时用，不用于生产。

## 4.2 UndefinedBehaviorSanitizer (UBSan)

**原理：**编译时插入检查，捕获 C 标准中定义为"未定义行为"的操作。

**启用：**

```
gcc -fsanitize=undefined -g -O1 program.c -o program
```



可以和 ASan 一起用：

```
gcc -fsanitize=address,undefined -g -O1 program.c -o program
```

### 它检测什么：

- 整数溢出 (`INT_MAX + 1`) —— 需单独开启 `-fsanitize=signed-integer-overflow`
- 除零
- NULL 指针解引用
- 对齐错误
- 移位溢出 (`1 << 31` 当 int 是 32 位时)
- 数组越界 (变长数组)

**面试常考：**"整数溢出是未定义行为吗？"

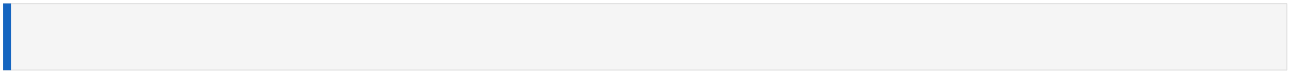
回答：signed 整数溢出是未定义行为 (UB)，unsigned 整数溢出是定义好的 (wraparound，模运算)。这就是为什么计数器、长度字段应该用 unsigned。

## 4.3 Valgrind (Memcheck)

**原理：**在**没有源码修改**的情况下，通过模拟 CPU 执行来追踪每块内存的状态。它给每个字节维护一个 "V-bit" (valid bit)，标记这个字节是否被初始化。

**启用：**

```
valgrind --leak-check=full --track-origins=yes ./program
```



|       |
|-------|
| 常用参数： |
|       |
|       |

`--leak-check=full` : 内存泄漏的完整报告 (哪里分配, 多少字节)

`--track-origins=yes` : 追踪未初始化值的来源（慢，但精准）

--show-leak-kinds=all : 显示所有类型的泄漏

`--log-file=vg.log` : 输出到文件

```
--num-callers=50      : 更深的调用栈
```

输出示例：

```
==54321== Conditional jump or move depends on uninitialised value(s)

==54321==      at 0x...: main (uninit.c:8)

==54321== Uninitialised value was created by a heap allocation

==54321==      at 0x...: malloc

==54321==      by 0x...: main (uninit.c:6)


==54321== 16 bytes in 1 blocks are definitely lost in loss record 1 of 2

==54321==      at 0x...: malloc

==54321==      by 0x...: leak_func (leak.c:5)

==54321==      by 0x...: main (leak.c:10)
```

4.4 三件武器对比

| 维度     | ASan  | UBSan | Valgrind     |
|--------|-------|-------|--------------|
| 检测方式   | 编译时插桩 | 编译时插桩 | 运行时模拟执行      |
| 需要重新编译 | ✔ 是   | ✔ 是   | ✘ 否（但建议加 -g） |
| 性能开销   | ~2x   | ~1.2x | ~10-50x      |



|         |      |       |      |
|---------|------|-------|------|
| 检测堆溢出   | ✓    | ✗     | ✓    |
| 检测栈溢出   | ✓    | ✗     | ✗    |
| 检测 UAF  | ✓    | ✗     | ✓    |
| 检测未初始化读 | ✗ 部分 | ✗     | ✓    |
| 检测整数溢出  | ✗    | ✓     | ✗    |
| 检测内存泄漏  | ✓    | ✗     | ✓    |
| 最适合场景   | 功能测试 | CI 编译 | 夜间测试 |

### 实战建议：

CI 流水线的最佳实践：

1. 编译时开启 ASan+UBSan `gcc -fsanitize=address,undefined`
2. 跑单元测试（如果 ASan 报错，测试直接 fail）
3. 用 Valgrind 跑一轮（较慢，但能发现 ASan 漏掉的未初始化读）
4. 两个都通过 = 内存安全置信度很高

## 第五阶段：实战任务

## 任务一：memory-lab（编码 A, ~3h）

写一个 C 程序，包含以下**独立的测试函数**，每个函数演示一类内存缺陷。每个函数必须：

1. 有清晰的注释说明 Bug 是什么
2. 写了"错误的版本"和"修复后的版本"（用 `#ifdef FIX` 或注释切换）
3. 运行后能用 ASan 或 Valgrind 检测到

**必须包含的缺陷类型：**

```
memory-lab/

├─ Makefile          # 至少三个 target: test-asan, test-valgrind, test-ubsan
├─ overflow.c        # 栈溢出 + 堆溢出 + off-by-one
├─ use_after_free.c  # UAF（基础版 + 别名指针版）
├─ memory_leak.c     # 泄漏（循环泄漏 + 提前返回泄漏）
├─ uninit.c          # 未初始化读（栈 + 堆）
├─ double_free.c     # 双重释放
└─ README.md         # 每类缺陷的 ASan/Valgrind 输出截图说明
```

**Makefile 要求：**

```
# 分别构建带 ASan 和普通版本

CC = gcc

CFLAGS = -Wall -Wextra -g -O1
```

```
asan: CFLAGS += -fsanitize=address

asan: all

valgrind: all

    valgrind --leak-check=full --track-origins=yes ./program

all: program
```

## 任务二：设计测试触发间歇性 Bug（实验，~2.5h）

**背景：**内存 Bug 最可怕的是"在开发机上跑 100 次都不挂，在客户那里第一次就挂"。

**要求：**

1. 写一个故意有缓冲区溢出的程序（溢出一个字节）
2. 写一个 shell 脚本循环跑这个程序 100 次
3. 记录 crash 的次数
4. 解释：为什么不是每次都 crash？（提示：内存对齐、栈布局）
5. 用 ASan 跑同样的循环，对比结果

## 任务三：阅读 Valgrind 报告并修复（编码 B，~3h）

我给你一段有多个内存 Bug 的代码（在 `broken-server.c` 中）。你需要：

1. 用 Valgrind 跑，理解每一条报告
2. 分类：哪些是 "definitely lost"，哪些是 "possibly lost"，哪些是 "still reachable"
3. 逐一修复所有 Bug
4. 修复后用 Valgrind 验证： "All heap blocks were freed -- no leaks are possible"

---

## 参考速查表

---

### ASan 错误类型

| 错误消息                   | 含义                                 |
|------------------------|------------------------------------|
| heap-buffer-overflow   | 堆缓冲区溢出（读写超出 malloc 的范围）            |
| stack-buffer-overflow  | 栈缓冲区溢出                             |
| heap-use-after-free    | 使用已释放的堆内存                          |
| stack-use-after-free   | 使用已释放的栈内存（如返回局部变量地址）               |
| double-free            | 双重释放                               |
| alloc-dealloc-mismatch | malloc 用 delete 释放 或 new 用 free 释放 |
| memory-leak            | 内存泄漏                               |

## Valgrind 泄漏分类

| 分类              | 含义                 | 严重程度            |
|-----------------|--------------------|-----------------|
| definitely lost | 没有任何指针指向这块内存       | ● 必须修           |
| indirectly lost | 指针本身在另一块 lost 内存里  | ● 修复直接泄漏后通常自动消失 |
| possibly lost   | 有指针指向内存中间（不是开头）    | ● 需要检查          |
| still reachable | 程序结束时还有指针指向但没 free | ● 可接受（但最好修）     |

## Valgrind 常见错误

| 错误消息                                            | 含义                        |
|-------------------------------------------------|---------------------------|
| Invalid read/write of size N                    | 访问了不该访问的内存（越界、UAF）        |
| Conditional jump depends on uninitialised value | if/循环条件用了未初始化的值           |
| Use of uninitialised value of size N            | 计算中使用了未初始化值               |
| Mismatched free/delete                          | malloc 和 free 不配对         |
| Source and destination overlap in memcpy        | memcpy 用于重叠区域（应用 memmove） |

---

## 常见面试追问

---

### 1. "ASan 和 Valgrind 的区别是什么？"

**标准回答：**

- ASan 是**编译时**插入的检查代码，运行时直接报错。需要重新编译，但快（~2x）。
- Valgrind 是**运行时**模拟执行，不需要重新编译但非常慢（~10-50x）。
- ASan 能检测栈溢出（Valgrind 不行），Valgrind 能检测未初始化读（ASan 不行）。
- 团队实践中两者互补：ASan 跑 CI 测试，Valgrind 跑夜间测试。

### 2. "什么样的内存错误最危险？为什么？"

**标准回答：**Use-After-Free。因为：

- 不一定 crash（不像 NULL 解引用那样必崩）
- 被 free 的内存可能被下一个 malloc 复用
- 攻击者可以通过堆布局控制被复用的内容
- 这就是各类浏览器漏洞（Chrome、Safari）最常见的类型

### 3. "为什么内存泄漏在某些场景下是安全风险？"

**标准回答：**CVE 中有"内存耗尽导致拒绝服务"的分类。对于网关/服务器这类长期运行的程序，持续的内存

泄漏会导致 OOM Killer 杀死进程，造成服务中断。此外，如果泄漏的内存包含敏感信息（密钥、token），而内存没有被清零，其他进程可能通过 `/proc//mem` 读取。

#### 4. "如何在没有 Valgrind 的生产环境定位内存问题？"

标准回答：

- 用 ASan 编译 debug 版部署到灰度环境
- 观察 `pmap -x` 看内存持续增长 → 疑似泄漏
- `strace -e brk,mmap` 看系统调用频率 → 频繁分配
- `/proc//maps` 看内存映射
- GDB + 自定义 malloc 钩子（`__malloc_hook`, `__free_hook`）追踪分配
- 或在代码中嵌入 jemalloc / tcmalloc 的 profiling 功能

#### 5. "解释一下 Stack buffer overflow 为什么可能导致任意代码执行？"

标准回答：

栈上除了局部变量，还保存了**返回地址**。如果溢出能够覆盖到返回地址（越过栈保护金丝雀 canary），就可以让函数返回时跳转到攻击者控制的地址（如指向 shellcode 或 ROP 链）。这正是栈溢出漏洞攻击的原理。现代防御包括：Stack Canary（`-fstack-protector`）、NX（栈不可执行）、ASLR（地址随机化）。

---

## 验收清单

---

- ☐ 能写出五类内存缺陷各自的最小复现代码
  - ☐ 每个缺陷都用 ASan 跑过，能看懂 ASan 的错误输出
  - ☐ 每个缺陷都用 Valgrind 跑过，能看懂 Valgrind 的输出
  - ☐ 能解释：为什么同一个溢出 Bug 可能有时 crash 有时不 crash
  - ☐ 能区分 Valgrind 的四种泄漏分类
  - ☐ 能说出 ASan 和 Valgrind 各自能检测和不能检测什么
  - ☐ broken-server.c 被成功修复，Valgrind 验证无泄漏
- 

> 📅 17 **下一周**：W3 — 现代 C++ 与 RAII（智能指针、移动语义、异常安全）

>

> 🤖 本教程由 Claude Code 作为技术导师编写 • sys-netsec-lab